

Module 02: Linked List, Stack, Queue

Language: Java**Type:** Pre-lab**Mode:** individual practical work

Learning outcomes

By completing this module, you should be able to:

1. Use LinkedList to store and manage a collection of data.
2. Use Queue to process data according to the FIFO (First In First Out) principle.
3. Use Stack to store and retrieve data according to the LIFO (Last In First Out) principle.
4. Read structured data from a text file using Scanner.
5. Process data sequentially by combining LinkedList, Queue, and Stack.
6. Explain how each data structure is used to solve a specific part of a problem.

Concept Toolkit

Concept	Meaning and a useful distinction
LinkedList	A linear data structure whose elements are connected sequentially. It can be used to store, add, remove, and access data in an ordered collection.
Queue	A data structure that follows the FIFO (First In First Out) principle. The element that enters first is processed first.
Stack	A data structure that follows the LIFO (Last In First Out) principle. The element added most recently is retrieved first.
poll()	Removes and returns the element at the front of a queue. It follows the FIFO principle. If the queue is empty, poll() returns null.
push()	Adds an element to the top of a stack. The newly added element becomes the first element to be retrieved by pop().
pop()	Removes and returns the element at the top of a stack. It follows the LIFO principle.
peek()	Returns the element at the front of a queue or the top of a stack without removing it.

Prelab Case: Bank Transaction Processing

A bank needs a simple program to process customer transactions. Transactions are recorded in a text file named **transactions.txt**. Each transaction contains the **customer's name**, the **type of transaction**, and the **amount of money** involved.

The bank processes transactions in the **same order in which they appear in the input data**. A deposit increases the customer's balance, while a withdrawal can only be completed when the customer's balance is sufficient. If a withdrawal cannot be completed because of insufficient balance, the transaction must be recorded as a **failed transaction**.

The program must use different data structures for different purposes:

- **LinkedList** stores **all transactions** read from the file.
- Another **LinkedList** stores **customer data** and their **current balances**.
- **Queue** processes **transactions** in FIFO order.
- **Stack** stores **withdrawal transactions that fail** because of insufficient balance.

Note: You do not need to create separate Transaction or Customer classes. Use simple structures such as String[] so that the main focus remains on understanding LinkedList, Queue, and Stack.

Exact Behaviour

- Read all transaction data from transactions.txt using Scanner.
- Each line contains three space-separated values: **<NAME> <TYPE> <AMOUNT>**
 - <NAME> represents the customer's name.
 - <TYPE> can be DEPOSIT or WITHDRAW.
 - <AMOUNT> is a positive integer representing the transaction amount.
- Store all transactions in a LinkedList.
- Customer data must also be stored in a LinkedList.
- Every customer's initial balance is 0.
- Customers must appear in the customer LinkedList according to the order in which they first appear in the transaction data.
- Move the transactions from the LinkedList into a Queue.
- Process transactions from the Queue using FIFO order.
- For DEPOSIT, add the transaction amount to the customer's balance.
- For WITHDRAW:
 - If the withdrawal amount is greater than the customer's current balance, the transaction fails and the balance does not change.
 - Otherwise, subtract the withdrawal amount from the customer's balance.
- Store every failed WITHDRAW transaction in a Stack.
- After all transactions have been processed, display:
 - The final balance of every customer.
 - Every failed withdrawal transaction.

Prelab Tasks

A1. Implementation and demonstration

Create a Java program that processes bank transactions using **LinkedList**, **Queue**, and **Stack**, following these steps:

1. **Read and store transactions** — Create a `LinkedList<String[]>` to store all transactions. Read `transactions.txt` using `Scanner`. For each line, parse the customer's name, transaction type, and amount, and store it as a `String[]`, preserving the original order.
2. **Create customer data** — Create another `LinkedList<String[]>` to store customer records (name and current balance). Set every customer's initial balance to 0. Add a customer to this list only the first time their name appears in the transaction data, preserving that first-appearance order. Do not add the same customer twice.
3. **Process transactions using Queue** — Create a `Queue<String[]>` and move all transactions from the transaction `LinkedList` into it. Process the queue in FIFO order: for each transaction, find the corresponding customer and update their balance.
4. **Store failed transactions using Stack** — Create a `Stack<String[]>`. Whenever a `WITHDRAW` fails, push it onto the stack. After all transactions are processed, retrieve and display the failed transactions in LIFO order (as produced by the stack).
5. Finally, **display the final balance of every customer and every failed withdrawal transaction**, matching the required output format.

A2. Required checks

Input file `transactions.txt`:

```
Raissa DEPOSIT 100000
Fachriza DEPOSIT 80000
Raissa WITHDRAW 30000
Dedy DEPOSIT 100000
Fachriza WITHDRAW 100000
Raissa DEPOSIT 50000
Dedy WITHDRAW 150000
Raissa DEPOSIT 150000
```

For the given `transactions.txt`, the transactions are processed in this order:

Transaction	Balance after processing
Raissa DEPOSIT 100000	Raissa = 100000
Fachriza DEPOSIT 80000	Fachriza = 80000
Raissa WITHDRAW 30000	Raissa = 70000
Dedy DEPOSIT 100000	Dedy = 100000

Labwork 02

Fachriza WITHDRAW 100000	Failed, Fachriza remains 80000
Raissa DEPOSIT 50000	Raissa = 120000
Dedy WITHDRAW 150000	Failed, Dedy remains 100000
Raissa DEPOSIT 150000	Raissa = 270000

The program execution must display exactly:

```
=== Final Balances ===
Raissa : 270000
Fachriza : 80000
Dedy : 100000

=== Failed Transactions ===
Dedy WITHDRAW 150000
Fachriza WITHDRAW 100000
```

Submission

Please follow these instructions carefully to submit your work:

1. Create the **lw02** folder under your **dsa-[nrp]** repository. Inside the **lw02** folder, create a subfolder named **prelab**.
2. Complete all of your work for this prelab inside the **prelab** folder.
3. Commit and push your work to GitHub with the commit message:
"**Labwork 02 - Prelab**"
4. **Deadline: Monday, September 28, 2026, at 6:00 AM (WIB).**

To give a clearer picture, The repository for this semester's practicum will follow the structure below:

